

AI IT Help Desk Assistant

Testing and Results Report

CMSC 495 - Group 2
Cory Allen • Kayden Ayers • Rich Sterchele

Purpose. This report consolidates testing performed during development and integration of the Java Swing/MySQL/OpenAI help desk application. It includes setup diagnostics, controlled reliability and performance tests, automated unit tests, GitHub Actions continuous integration results, and JaCoCo code-coverage measurements.

Interpretation note. Diagnostic failures encountered while configuring external dependencies are listed separately from controlled reliability measurements. A database attempt made while MySQL was not running, for example, was a setup failure and was not counted as a failed trial in the 10-run reliability test.

Area	Final measured result
Database connection reliability	10/10 successful (100%) with MySQL available
Database connection performance	281.80 ms average; 247.93-317.28 ms observed range
Ticket creation	10/10 successful (100%)
Ticket retrieval/display	10/10 successful (100%)
Ticket status updates	10/10 successful (100%)
Automated checks	11/11 passed (100%)
GitHub Actions CI	PASS
JaCoCo coverage generation	PASS
TriageResult production class	100% instruction, line, and method coverage
TicketTriageService production class	76.7% line; 80.7% instruction; 90% branch coverage

1. Test Environment and Integration Diagnostics

The application was tested as a Java desktop program using IntelliJ IDEA on Windows. It uses Java Swing for the GUI, JDBC for database access, a local MySQL/MariaDB database through XAMPP, MySQL Connector/J, and an external OpenAI API integration.

1.1 Database setup diagnostics

Diagnostic	Observed result	Resolution / meaning
JDBC connection before driver setup	No suitable driver found	MySQL Connector/J was added to the IntelliJ module dependencies.
Connection after JDBC driver installation but before MySQL service	CommunicationsException / Connection refused	XAMPP MySQL service was started.
Initial database selection	Unknown database	helpdesk_db was created/imported and tables were verified.
Final DatabaseConnection verification	Database connection successful	Confirmed Java-to-MySQL connectivity before controlled trials.

1.2 AI/API integration diagnostics

Test / observation	Result
ApiKeyTest	Initially reported OPENAI_API_KEY missing; the environment variable was then configured for the relevant run context.
OpenAI/Triage request	AI triage initially returned blank structured fields.
Raw API response inspection	HTTP 429 with credit_balance_exhausted / insufficient quota was identified.
Error-handling correction	OpenAIService was changed to throw/report non-2xx HTTP responses instead of passing error JSON to the triage parser.
Post-fix behavior	The actual HTTP 429 error was surfaced instead of misleading blank triage output.

The HTTP 429 result reflects the external API project's billing/quota state and was not counted as an application reliability failure. The error-handling test demonstrated that the application exposes the real external-service failure rather than silently treating it as a valid AI response.

2. Controlled Database Reliability and Performance Testing

With MySQL running and the database correctly configured, DatabaseConnection was executed 10 times. All 10 controlled trials completed successfully.

Metric	Result
Successful connections	10/10
Connection success rate	100%
Failed controlled trials	0/10

2.1 Database connection timing

Connection time was measured using System.nanoTime() around DatabaseConnection.getConnection().

Trial	Connection time (ms)
1	247.93
2	310.29
3	317.28
4	300.42
5	268.94
6	261.98
7	264.75
8	278.16
9	258.57
10	309.65

Statistic	Value
Average	281.80 ms
Fastest	247.93 ms
Slowest	317.28 ms
Observed range	69.35 ms

These measurements describe local test conditions and should not be interpreted as a universal production-performance guarantee.

3. End-to-End Ticket Workflow Reliability

A controlled set of 10 Performance Test Tickets exercised the core database-backed workflow.

Workflow operation	Trials	Successful	Rate
Ticket creation	10	10	100%
Ticket retrieval/display	10	10	100%
Ticket status update: Open to In Progress	10	10	100%

Ticket creation exercised Swing input handling, validation, JDBC, ticket insertion, ticket-history insertion, and transaction commit. All 10 created tickets were subsequently retrieved and displayed. Each was then successfully changed from Open to In Progress.

These results support reliability under the controlled local test conditions used for the project; they do not establish 100% reliability under every possible environment, load, or failure condition.

4. Automated Unit Testing

Two lightweight Java test classes were configured to return a nonzero process exit code if an automated check fails, allowing GitHub Actions to enforce them during continuous integration.

4.1 TriageResultTest

Automated check	Result
Category getter	PASS
Priority getter	PASS
Summary getter	PASS
Department getter	PASS
toString output	PASS

Result: 5/5 checks passed (100%).

4.2 TicketTriageServiceTest

Automated check	Result
Valid response category	PASS
Valid response priority	PASS
Valid response summary	PASS
Valid response department	PASS
Whitespace is trimmed	PASS
Missing department produces empty value	PASS

Result: 6/6 checks passed (100%). **Combined result:** 11/11 automated checks passed.

5. GitHub Actions Continuous Integration Testing

The GitHub Actions workflow checks out the repository, configures Temurin Java 21, downloads MySQL Connector/J and JaCoCo, compiles the Java source, runs the automated tests, generates coverage reports, uploads the coverage report as an artifact, and verifies compiled classes.

CI evidence	Result
Initial compile-only Java CI run	PASS; approximately 14 seconds
CI after automated test integration	PASS
CI with JaCoCo integration	PASS; approximately 15 seconds
Intermediate run before new Java test source was pushed	FAIL as expected; superseded by integrated run
Final integrated run with both test suites	PASS; approximately 11-13 seconds
Automated checks in integrated run	5/5 + 6/6 = 11/11 PASS
Coverage report generation	PASS
Coverage artifact upload	PASS

The temporary failed run occurred because the workflow referenced TicketTriageServiceTest before its Java source changes were pushed. Once the source changes were integrated, the next run passed. This demonstrated that the pipeline failed when the repository was inconsistent and passed after the required source was present.

6. JaCoCo Code Coverage Results

JaCoCo generated HTML, XML, and CSV coverage reports. Test pass rate and coverage measure different things: 11/11 checks passing means all defined checks succeeded, while coverage shows which production code was actually executed.

6.1 Baseline coverage

Production class	Instructions	Lines	Methods	Branches
TriageResult	37/37 covered (100%)	11/11 covered (100%)	6/6 covered (100%)	No branches
TicketTriageService	0/103 covered	0/27 covered	0/3 covered	0/10 covered

The baseline report showed that TriageResult was fully exercised, while TicketTriageService had not yet been exercised under JaCoCo. This identified a measurable testing gap.

6.2 Coverage after adding TicketTriageServiceTest

Production class	Line coverage	Instruction coverage	Branch coverage	Method coverage
TriageResult	100%	100%	N/A - no branches	100%
TicketTriageService	23/30 = 76.7%	88/109 = 80.7%	9/10 = 90%	2/4 = 50%

The TicketTriageService test deliberately exercises deterministic response-parsing logic without making a live OpenAI request. The remaining uncovered production paths include the normal live-service construction/request path. This progression - measuring coverage, identifying an untested class, adding targeted tests, and re-measuring - provides evidence of iterative quality improvement.

The earlier 88.5% figure belongs to TriageResultTest, the test class, not the production TriageResult class. Production TriageResult is fully covered in the supplied JaCoCo CSV.

7. Consolidated Results and Conclusions

Evidence category	Final result
Database connectivity	10/10 successful controlled connections (100%)

Evidence category	Final result
Database connection speed	281.80 ms average; 247.93-317.28 ms observed
Ticket creation	10/10 successful
Ticket retrieval/display	10/10 successful
Ticket status updates	10/10 successful
Automated checks	11/11 passed
Continuous integration	Final integrated GitHub Actions run PASS
TriageResult production coverage	100% instructions, lines, and methods
TicketTriageService production coverage	76.7% lines; 80.7% instructions; 90% branches
External AI error handling	HTTP 429 quota error correctly surfaced after non-2xx handling fix

Overall, the testing combined configuration diagnostics, controlled integration trials, automated checks, continuous integration, and code-coverage measurement. The strongest evidence is not a claim of universal reliability, but repeatable successful behavior under controlled conditions together with automated verification of deterministic AI-response parsing and explicit handling of external-service failures.

8. Evidence to Preserve for Final Submission

- Successful GitHub Actions integrated CI run showing compilation, both automated test suites, JaCoCo report generation, artifact upload, and verification.
- Expanded CI output showing Tests passed: 5/5 and Tests passed: 6/6.
- JaCoCo HTML/CSV reports showing baseline and final production-class coverage.
- Final end-to-end demonstration evidence after the team's final program files are integrated.